



Security Review For Lombard Finance



Collaborative Audit Prepared For:
Lead Security Expert(s):

Lombard Finance
carrotsmuggler

Date Audited:

g
March 9 - March 20, 2026

Introduction

This audit covers the Solana program changes that replace the legacy LBTC mint flow with a Generalized Message Passing (GMP) bus and add NativeLBTC (BTC.b) support.

The new mailbox program exposes each outbound payload as a program-derived account seeded by a monotonic nonce so the Ledger notary can fetch and notarize it, and a mailbox_receiver program delivers inbound consortium-signed payloads to handler programs through a CPI whose signer constraint ties the payload account to the configured mailbox. A bascule_gmp program ports the EVM Bascule allowlist to Solana as a threshold of MintValidator signatures recorded on a per-mint PDA, and is CPI'd from asset_router's gmp_receive before any mint. The asset_router program holds a single native_mint (BTC.b) plus per-token TokenConfig and per-route TokenRoute PDAs, exposes deposit/redeem/redeem_for_btc instructions that emit GMP messages via mailbox, and routes inbound mints through bascule_gmp. The bridge program handles GMP-based cross-chain transfers with its own remote-bridge/remote-token configs and rate limiter. The consortium program substitutes the mint-validation flow present in legacy LBTC with a more general validation of all messages signed by the Ledger consortium participants, and is used by the flows listed above; lombard_token_pool is adjusted for the additional token context in CCIP transfers.

Scope

Repository: lombard-finance/sol-contracts

Audited Commit: 79025376bee50b9ce08c841a631986567607975c

Final Commit: 21d81cff91da2632c83f91ce7a21c87facdceb7f

Files:

- programs/asset_router/src/constants.rs
- programs/asset_router/src/errors.rs
- programs/asset_router/src/events.rs
- programs/asset_router/src/instructions/accept_ownership.rs
- programs/asset_router/src/instructions/admin.rs
- programs/asset_router/src/instructions/change_auth.rs
- programs/asset_router/src/instructions/change_native_mint.rs
- programs/asset_router/src/instructions/deposit.rs
- programs/asset_router/src/instructions/gmp_receive.rs
- programs/asset_router/src/instructions/grant_account_role.rs
- programs/asset_router/src/instructions/initialize.rs
- programs/asset_router/src/instructions/mint_from_payload.rs

- `programs/asset_router/src/instructions/mint_with_fee.rs`
- `programs/asset_router/src/instructions/mod.rs`
- `programs/asset_router/src/instructions/pause.rs`
- `programs/asset_router/src/instructions/redeem_for_btc.rs`
- `programs/asset_router/src/instructions/redeem.rs`
- `programs/asset_router/src/instructions/revoke_account_roles.rs`
- `programs/asset_router/src/instructions/set_mint_fee.rs`
- `programs/asset_router/src/instructions/set_token_config.rs`
- `programs/asset_router/src/instructions/set_token_route.rs`
- `programs/asset_router/src/instructions/set_treasury.rs`
- `programs/asset_router/src/instructions/unset_token_route.rs`
- `programs/asset_router/src/lib.rs`
- `programs/asset_router/src/security.rs`
- `programs/asset_router/src/state.rs`
- `programs/asset_router/src/utils/bitcoin_utils.rs`
- `programs/asset_router/src/utils/consortium_payloads.rs`
- `programs/asset_router/src/utils/ed25519.rs`
- `programs/asset_router/src/utils/fee.rs`
- `programs/asset_router/src/utils/gmp_messages.rs`
- `programs/asset_router/src/utils/mod.rs`
- `programs/bascule_gmp/src/constants.rs`
- `programs/bascule_gmp/src/errors.rs`
- `programs/bascule_gmp/src/events.rs`
- `programs/bascule_gmp/src/instructions/accept_ownership.rs`
- `programs/bascule_gmp/src/instructions/admin.rs`
- `programs/bascule_gmp/src/instructions/grant_account_role.rs`
- `programs/bascule_gmp/src/instructions/initialize.rs`
- `programs/bascule_gmp/src/instructions/mod.rs`
- `programs/bascule_gmp/src/instructions/pause.rs`
- `programs/bascule_gmp/src/instructions/report_mint.rs`
- `programs/bascule_gmp/src/instructions/revoke_account_roles.rs`

- programs/bascul_gmp/src/instructions/update_validate_threshold.rs
- programs/bascul_gmp/src/instructions/validate_mint.rs
- programs/bascul_gmp/src/lib.rs
- programs/bascul_gmp/src/security.rs
- programs/bascul_gmp/src/state.rs
- programs/bridge/src/constants.rs
- programs/bridge/src/errors.rs
- programs/bridge/src/events.rs
- programs/bridge/src/instructions/accept_ownership.rs
- programs/bridge/src/instructions/admin.rs
- programs/bridge/src/instructions/deposit.rs
- programs/bridge/src/instructions/gmp_receive.rs
- programs/bridge/src/instructions/grant_account_role.rs
- programs/bridge/src/instructions/initialize.rs
- programs/bridge/src/instructions/mod.rs
- programs/bridge/src/instructions/pause.rs
- programs/bridge/src/instructions/revoke_account_roles.rs
- programs/bridge/src/instructions/set_local_token_config.rs
- programs/bridge/src/instructions/set_rate_limit.rs
- programs/bridge/src/instructions/set_remote_bridge_config.rs
- programs/bridge/src/instructions/set_remote_token_config.rs
- programs/bridge/src/instructions/set_sender_config.rs
- programs/bridge/src/instructions/unset_local_token_config.rs
- programs/bridge/src/instructions/unset_remote_bridge_config.rs
- programs/bridge/src/instructions/unset_remote_token_config.rs
- programs/bridge/src/instructions/unset_sender_config.rs
- programs/bridge/src/lib.rs
- programs/bridge/src/state.rs
- programs/bridge/src/utils/gmp_messages.rs
- programs/bridge/src/utils/mod.rs
- programs/bridge/src/utils/token_actions.rs

- `programs/consortium/src/constants.rs`
- `programs/consortium/src/errors.rs`
- `programs/consortium/src/events.rs`
- `programs/consortium/src/instructions/accept_ownership.rs`
- `programs/consortium/src/instructions/admin.rs`
- `programs/consortium/src/instructions/create_session.rs`
- `programs/consortium/src/instructions/finalize_session.rs`
- `programs/consortium/src/instructions/initialize.rs`
- `programs/consortium/src/instructions/mod.rs`
- `programs/consortium/src/instructions/post_session_payload.rs`
- `programs/consortium/src/instructions/post_session_signatures.rs`
- `programs/consortium/src/instructions/set_initial_valset_from_session.rs`
- `programs/consortium/src/instructions/set_initial_valset.rs`
- `programs/consortium/src/instructions/update_valset.rs`
- `programs/consortium/src/lib.rs`
- `programs/consortium/src/security.rs`
- `programs/consortium/src/state.rs`
- `programs/consortium/src/utils/mod.rs`
- `programs/consortium/src/utils/session_payloads.rs`
- `programs/consortium/src/utils/signatures.rs`
- `programs/lombard_token_pool/build.rs`
- `programs/lombard_token_pool/src/constants.rs`
- `programs/lombard_token_pool/src/errors.rs`
- `programs/lombard_token_pool/src/events.rs`
- `programs/lombard_token_pool/src/instructions/accept_ownership.rs`
- `programs/lombard_token_pool/src/instructions/admin.rs`
- `programs/lombard_token_pool/src/instructions/append_remote_pool_addresses.rs`
- `programs/lombard_token_pool/src/instructions/configure_allow_list.rs`
- `programs/lombard_token_pool/src/instructions/delete_chain_config.rs`
- `programs/lombard_token_pool/src/instructions/edit_configs.rs`
- `programs/lombard_token_pool/src/instructions/initialize.rs`

- `programs/lombard_token_pool/src/instructions/lock_or_burn_tokens.rs`
- `programs/lombard_token_pool/src/instructions/mod.rs`
- `programs/lombard_token_pool/src/instructions/release_or_mint_tokens.rs`
- `programs/lombard_token_pool/src/instructions/remove_from_allow_list.rs`
- `programs/lombard_token_pool/src/instructions/set_chain_rate_limit.rs`
- `programs/lombard_token_pool/src/instructions/set_config.rs`
- `programs/lombard_token_pool/src/instructions/set_rate_limit_admin.rs`
- `programs/lombard_token_pool/src/lib.rs`
- `programs/lombard_token_pool/src/state.rs`
- `programs/mailbox/src/constants.rs`
- `programs/mailbox/src/errors.rs`
- `programs/mailbox/src/events.rs`
- `programs/mailbox/src/instructions/accept_ownership.rs`
- `programs/mailbox/src/instructions/admin.rs`
- `programs/mailbox/src/instructions/deliver_message.rs`
- `programs/mailbox/src/instructions/disable_inbound_message_path.rs`
- `programs/mailbox/src/instructions/disable_outbound_message_path.rs`
- `programs/mailbox/src/instructions/enable_inbound_message_path.rs`
- `programs/mailbox/src/instructions/enable_outbound_message_path.rs`
- `programs/mailbox/src/instructions/grant_account_role.rs`
- `programs/mailbox/src/instructions/handle_message.rs`
- `programs/mailbox/src/instructions/initialize.rs`
- `programs/mailbox/src/instructions/mod.rs`
- `programs/mailbox/src/instructions/pause.rs`
- `programs/mailbox/src/instructions/revoke_account_roles.rs`
- `programs/mailbox/src/instructions/send_message.rs`
- `programs/mailbox/src/instructions/set_sender_config.rs`
- `programs/mailbox/src/instructions/unset_sender_config.rs`
- `programs/mailbox/src/lib.rs`
- `programs/mailbox/src/security.rs`
- `programs/mailbox/src/state.rs`

- programs/mailbox/src/utils/cpi.rs
- programs/mailbox/src/utils/message_utils.rs
- programs/mailbox/src/utils/mod.rs
- programs/ratio_oracle/src/constants.rs
- programs/ratio_oracle/src/errors.rs
- programs/ratio_oracle/src/events.rs
- programs/ratio_oracle/src/instructions/accept_ownership.rs
- programs/ratio_oracle/src/instructions/admin.rs
- programs/ratio_oracle/src/instructions/initialize_oracle.rs
- programs/ratio_oracle/src/instructions/initialize.rs
- programs/ratio_oracle/src/instructions/mod.rs
- programs/ratio_oracle/src/instructions/publish_ratio.rs
- programs/ratio_oracle/src/instructions/update_ratio_threshold.rs
- programs/ratio_oracle/src/lib.rs
- programs/ratio_oracle/src/security.rs
- programs/ratio_oracle/src/state.rs
- programs/ratio_oracle/src/utils/consortium_payloads.rs
- programs/ratio_oracle/src/utils/mod.rs
- programs/ratio_oracle/src/utils/ratio_threshold.rs

Final Commit Hash

21d81cff91da2632c83f91ce7a21c87facdceb7f

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
4	3	1

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: Lacking verification that the instruction is from the Ed25519 program allows signature bypass and arbitrary fee charges [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-03-lombard-solana-contracts/issues/123>

Summary

`verify_ed25519_instruction` does not verify that the prior instruction is actually dispatched to Solana's Ed25519 program.

Because only instruction data is parsed (without checking `program_id`), an attacker can provide a non-Ed25519 instruction with crafted bytes that satisfy local parsing checks.

Vulnerability Detail

In `asset_router::utils::ed25519::verify_ed25519_instruction`, the code loads the previous instruction from the instruction sysvar and assumes it is an Ed25519 verification instruction. It then validates fields (signature count, offsets, pubkey/message/signature bytes) directly from instruction data.

However, it never enforces:

- `ed25519_instruction.program_id == solana_program::ed25519_program::id()`

As a result, the function can return `Ok(())` even when no Ed25519 signature verification has actually run in the transaction. The check becomes a parser check over attacker-controlled bytes rather than a cryptographic verification guarantee.

Exploit / trigger path (practical):

1. Attacker submits a transaction that calls a flow relying on `verify_ed25519_instruction`.
2. Immediately before that call, attacker places an arbitrary instruction to any program they control (or another permissive program), not the Ed25519 precompile.
3. Attacker crafts this instruction's data so:
 - `num_signatures == 1`
 - offsets deserialize correctly
 - slices at computed offsets match the `expected_public_key`, message, and signature bytes expected by downstream logic.
4. `verify_ed25519_instruction` accepts the crafted instruction data and returns success.

5. Downstream logic proceeds under the false assumption that the Ed25519 program verified authenticity.

This is a real authentication bypass condition because the security invariant ("signature must be validated by the Ed25519 program") is not actually enforced.

Impact

- **Affected behavior:** authorization paths depending on this helper can be satisfied without cryptographic verification.
- **Security consequence:** attacker can forge acceptance of signed payloads and influence trusted parameters (including fee-related payload fields), enabling unauthorized state transitions and potential economic loss (e.g., arbitrary fee charging to mint recipients).

Code Snippet

```
// sol-contracts/programs/asset_router/src/utils/ed25519.rs
let ed25519_instruction =
    load_instruction_at_checked((current_index - 1) as usize, instruction_sysvar)?;

// Verify the content of the Ed25519 instruction
let instruction_data = ed25519_instruction.data;
// ...parsing and byte comparisons...
// Missing: verify `ed25519_instruction.program_id` is Ed25519 program id
```

Tool Used

Manual Review

Recommendation

Require `ed25519_instruction.program_id` to equal Solana's Ed25519 program id before parsing/accepting the instruction data.

Discussion

555-andrew

Acknowledged. Fixed in this pull request:

<https://github.com/lombard-finance/sol-contracts/pull/54>

Issue H-2: Missing check on `*_instruction_index` fields allow anyone to charge arbitrary mint fees to recipients [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-03-lombard-solana-contracts/issues/124>

Summary

`verify_ed25519_instruction` does not validate `signature_instruction_index`, `public_key_instruction_index`, and `message_instruction_index` from `Ed25519SignatureOffsets`. This allows an attacker to make local byte-slice checks pass while the Ed25519 precompile verifies different pubkey/message/signature bytes from other instructions in the same transaction.

Vulnerability Detail

The function assumes the verified signature material is contained in the current Ed25519 instruction data and only checks offsets into `instruction_data`. However, Solana's Ed25519 precompile supports cross-instruction references through the `*_instruction_index` fields.

Because these index fields are never constrained, an attacker can:

1. Build an Ed25519 instruction whose local data includes bytes that match the program's expected `expected_public_key`, `message`, and `signature` at the checked offsets.
2. Set one or more `*_instruction_index` fields to point to different instruction(s) containing attacker-controlled pubkey/message/signature bytes.
3. Have the Ed25519 precompile verify the referenced external bytes, not the locally checked bytes.
4. Pass `verify_ed25519_instruction` while bypassing the intended binding between "what was verified" and "what was parsed by this program".

This is a verification-confusion bug: local checks and precompile verification can refer to different data sources.

Impact

- **Affected behavior:** signature verification integrity in paths relying on `verify_ed25519_instruction`.

- **Security consequence:** authorized signer checks can be bypassed in practice, enabling unauthorized payload acceptance (including arbitrary fee charging to mint recipients as noted in code comments).

Code Snippet

```
// sol-contracts/programs/asset_router/src/utils/ed25519.rs
let offsets: Ed25519SignatureOffsets =
    Ed25519SignatureOffsets::try_from_slice(&instruction_data[2..16])?;

// No validation that these are u16::MAX (current instruction data)
let pubkey_start = offsets.public_key_offset as usize;
let pubkey_end = pubkey_start + 32;
if &instruction_data[pubkey_start..pubkey_end] != expected_public_key {
    return Err(AssetRouterError::InvalidPublicKey.into());
}

let msg_start = offsets.message_data_offset as usize;
let msg_end = msg_start + offsets.message_data_size as usize;
if &instruction_data[msg_start..msg_end] != message {
    return Err(AssetRouterError::InvalidMessage.into());
}

let sig_start = offsets.signature_offset as usize;
let sig_end = sig_start + 64;
if &instruction_data[sig_start..sig_end] != signature {
    return Err(AssetRouterError::InvalidSignature.into());
}
```

Tool Used

Manual Review

Recommendation

Require all three index fields (signature_instruction_index, public_key_instruction_index, message_instruction_index) to be u16::MAX so pubkey/message/signature are forced to come from the current Ed25519 instruction data.

Discussion

555-andrew

Acknowledged. Fixed in this pull request:

<https://github.com/lombard-finance/sol-contracts/pull/68>

Issue H-3: Inbound payload validation and processing can be blocked by sending lamports to ValidatedPayload PDAs [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-03-lombard-solana-contracts/issues/125>

Summary

`create_session` can be permanently blocked for a given `payload_hash` because it requires the `validated_payload` PDA to have exactly zero lamports before the account is initialized. Any external actor can transfer lamports to that PDA address, causing the instruction to fail and creating a denial of service for session creation and effectively blocking all inbound cross-chain messages.

Vulnerability Detail

In `CreateSession`, the `validated_payload` account is expected to not exist yet. The code enforces this with:

- `validated_payload.data_is_empty()`
- `validated_payload.lamports() == 0`
- `validated_payload.owner == System::id()`

For a PDA that has not been initialized, the address is still a valid system account target for plain SOL transfers. An attacker can send 1 lamport to the derived `validated_payload` PDA for a target `payload_hash`. This makes `validated_payload.lamports() != 0`, so `create_session` fails with `ValidatedPayloadAlreadyExists` even though no validated payload account data exists.

Because the PDA is deterministic (`[VALIDATED_PAYLOAD_SEED, payload_hash]`), this can be precomputed and grieved ahead of legitimate session creation.

Impact

Denial of service on session creation for targeted payloads. Attackers can block notary/session workflow progress by funding uninitialized `validated_payload` PDA addresses. In effect, no inbound messages can be validated and processed.

Code Snippet

```
/// CHECK: Only want to check that the validated payload account does not exist.
#[account(
```

```

    constraint = validated_payload.data_is_empty() @
    ↪ ConsortiumError::ValidatedPayloadNotEmpty,
    // @audit-issue payload validation can be blocked by sending lamports to the
    ↪ uninitialized PDA
    constraint = validated_payload.lamports() == 0 @
    ↪ ConsortiumError::ValidatedPayloadAlreadyExists,
    constraint = validated_payload.owner == &System::id() @
    ↪ ConsortiumError::ValidatedPayloadAlreadyExists,
    seeds = [VALIDATED_PAYLOAD_SEED, &payload_hash[..]],
    bump
  )]
pub validated_payload: UncheckedAccount<'info>,

```

Tool Used

Manual Review

Recommendation

Avoid using `lamports() == 0` as an existence check for PDAs. This constraint can be removed since the other constraints are enough to ensure that the account is not initialized.

Discussion

555-andrew

Acknowledged. Fixed here: <https://github.com/lombard-finance/sol-contracts/pull/68>

Issue H-4: Missing mut keyword leads to non-persistent changes [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-03-lombard-solana-contracts/issues/128>

Summary

In the `token_pool` contract, the admin is allowed to change certain parameters of the program. However, the related account is not marked mutable, leading to failed changes.

Vulnerability Detail

In the `set_router` and `set_rmn` instructions, the `state Account` is required, which contains the parameters. However, this account is not marked `mut`.

Due to this, any attempts to modify this account via these instructions will fail at runtime and the changes will not persist.

Impact

The admin will be unable to modify the router and rmn values.

Code Snippet

https://github.com/sherlock-audit/2026-03-lombard-solana-contracts/blob/def4cec29853df194a31e1fb69d92ac2572e861c/sol-contracts/programs/lombard_token_pool/src/instructions/admin.rs#L16-L21

Tool Used

Manual Review

Recommendation

Mark the account as `mut`.

Discussion

555-andrew

Acknowledged. Fixed here: <https://github.com/lombard-finance/sol-contracts/pull/68>

Issue M-1: Bascule GMP Mint validation breaks in Asset Router's gmp_receive due to incorrect validator account [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-03-lombard-solana-contracts/issues/122>

Summary

`gmp_receive` passes the wrong account as validator in the `bascule_gmp::validate_mint` CPI.

It uses `ctx.accounts.config` instead of the signer PDA (`bascule_validator`), causing the CPI signer check to fail whenever Bascule validation is enabled.

Vulnerability Detail

In `asset_router::gmp_receive`, the CPI is built with signer seeds for `BASCULE_VALIDATOR_SEED`, but the `ValidateMint.validator` account is set to `ctx.accounts.config`.

`bascule_gmp::ValidateMint` requires:

- `validator: Signer<'info>`
- `account_roles` derived from `[ACCOUNT_ROLES_SEED, validator.key().as_ref()]`

Because `config` is not signed (and is not the PDA represented by the provided signer seeds), the CPI fails the signer constraint before validation logic proceeds.

This creates a permanent failure path for `gmp_receive` when `config.bascule_gmp` is `Some(..)`.

Exploit / trigger path (practical):

1. `asset_router` is configured with a non-None `bascule_gmp` program.
2. Any valid inbound GMP mint message is processed through `gmp_receive`.
3. The function calls `validate_mint` CPI with:
 - signer seeds for `bascule_validator`
 - `validator = config` (non-signer in this CPI)
4. `bascule_gmp` rejects the CPI due to missing signer on validator.
5. `gmp_receive` reverts; mint cannot complete.

This is a real, deterministic denial-of-service condition on the mint-receive path under normal configuration.

Impact

- **Affected behavior:** all GMP mints routed through `gmp_receive` fail when Bascule validation is enabled.
- **Security consequence:** protocol-level liveness failure / DoS of cross-chain mint finalization, potentially leaving bridged user funds effectively stuck/unclaimable on Solana until fixed or reconfigured.

Code Snippet

```
// sol-contracts/programs/asset_router/src/instructions/gmp_receive.rs
validate_mint(
    CpiContext::new_with_signer(
        bascule_gmp_address.to_account_info(),
        ValidateMint {
            validator: ctx.accounts.config.to_account_info(), // wrong account
            config: bascule_gmp_config.to_account_info(),
            account_roles: bascule_gmp_account_roles.to_account_info(),
            mint_payload: bascule_gmp_mint_payload.to_account_info(),
            system_program: ctx.accounts.system_program.to_account_info(),
        },
        signer_seeds, // signs bascule_validator PDA, not config
    ),
    mint_message,
)?;
```

Tool Used

Manual Review

Recommendation

Pass `ctx.accounts.bascule_validator` as the CPI validator account (the same account represented by `signer_seeds`) so signer and role-derivation checks are consistent.

Discussion

555-andrew

Acknowledged. Fixed in this pull request:

<https://github.com/lombard-finance/sol-contracts/pull/70>

Issue M-2: Valid in-progress sessions are blocked if a new epoch increases its validators [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-03-lombard-solana-contracts/issues/126>

Summary

`post_session_signatures` resets `session.signed` to the current validator-set length when epoch changes, but the `Session` account is not reallocated. If the validator set has grown since session creation, this write exceeds the originally allocated account data and causes the instruction to fail.

Vulnerability Detail

A session is initialized in `create_session` with space derived from `Session::size(config.current_validators.len())`, so storage matches validator count at creation time.

Later, `post_session_signatures` calls `session.new_epoch(config.current_epoch, config.current_validators.len())` whenever session epoch is stale. `new_epoch` assigns `self.signed = vec![false; val_len]` unconditionally.

When `val_len` is greater than the original allocated capacity, Anchor/Borsh serialization cannot fit the updated vector into the existing account data. The transaction aborts, and signature posting for that session becomes impossible.

This is practically reachable whenever:

1. A user creates a session under epoch E with validator set size N .
2. Consortium rotates to epoch $E+1$ with size $N+k$ ($k > 0$).
3. User calls `post_session_signatures` on the old session.
4. Epoch reset path executes and fails during account serialization due to insufficient space.

Impact

- **Affected behavior:** session progression across epoch changes in `post_session_signatures`.
- **Security consequence:** incidental denial of service for affected sessions; they cannot collect signatures or be finalized once validator set growth triggers the stale-epoch reset path.

Code Snippet

```
// sol-contracts/programs/consortium/src/instructions/post_session_signatures.rs
if ctx.accounts.session.epoch != ctx.accounts.config.current_epoch {
    ctx.accounts.session.new_epoch(
        ctx.accounts.config.current_epoch,
        ctx.accounts.config.current_validators.len(),
    );
}
```

```
// sol-contracts/programs/consortium/src/state.rs
impl Session {
    pub fn new_epoch(&mut self, epoch: u64, val_len: usize) {
        self.epoch = epoch;
        self.signed = vec![false; val_len];
        self.weight = 0;
    }
}
```

Tool Used

Manual Review

Recommendation

Before increasing `session.signed` length, reallocate the `Session` account to at least `8 + Session::size(new_validator_len)` bytes.

Discussion

555-andrew

As a result of internal discussion we decide to utilize a different approach to address this issue. We will add epoch to the list of seeds for `Session` PDA and also provide function to close `Session` PDA for provided epoch. Fixed in these commits: <https://github.com/lombard-finance/sol-contracts/commit/8f39a610abfe7322f8eac5b016907e90896fff3b> <https://github.com/lombard-finance/sol-contracts/commit/293e0bae704224e1bc4ac15da170d0a90a1c3462> <https://github.com/lombard-finance/sol-contracts/commit/2b3f7e06e76412d398ec4f74f3fae81fd0021847>

Issue M-3: Enabling Mailbox fees for the Bridge sender blocks all LombardTokenPool deposits [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-03-lombard-solana-contracts/issues/127>

Summary

`lock_or_burn_tokens` always forwards `treasury`: `None` when calling `bridge::deposit`. If Mailbox fees are enabled for the Bridge sender, the CPI requires a treasury account to receive fee funds. Because this instruction does not allow callers to provide that account, all deposits through LombardTokenPool can be hard-blocked.

Vulnerability Detail

In `bridge_deposit_for_burn_with_caller`, the `Deposit` CPI accounts are constructed with:

- `sender_config` coming from `bridge_sender_config`
- `mailbox_sender_config` coming from `mailbox_sender_config`
- `treasury` hardcoded to `None`

When Bridge/Mailbox sender configuration enables fees, the downstream `deposit` flow expects a treasury account to be present and writable for fee handling. Since LombardTokenPool never supplies one, the CPI fails deterministically for every `lock_or_burn_tokens` call once fees are turned on.

This creates a protocol-level dependency bug: a valid configuration change in Bridge/Mailbox can permanently break LombardTokenPool deposits until code is upgraded.

Impact

Denial of service for cross-chain deposits/burns via LombardTokenPool:

- All `lock_or_burn_tokens` calls revert when sender fees are enabled.
- Users cannot bridge out through this pool.
- This can halt protocol functionality for the affected token/route until remediation.

Code Snippet

```
Deposit {  
  // ...  
  remote_token_config: ctx.accounts.remote_token_config.to_account_info(),
```

```
    treasury: None,  
    system_program: ctx.accounts.system_program.to_account_info(),  
}
```

Source:

[sol-contracts/programs/lombard_token_pool/src/instructions/lock_or_burn_tokens.rs](https://github.com/lombard-finance/sol-contracts/blob/master/programs/lombard_token_pool/src/instructions/lock_or_burn_tokens.rs)

Tool Used

Manual Review

Recommendation

Allow `lock_or_burn_tokens` callers (or config) to pass a validated treasury account to the Bridge `deposit` CPI so fee-enabled sender configurations remain compatible.

Discussion

555-andrew

Acknowledged. Fixed in this pull request:

<https://github.com/lombard-finance/sol-contracts/pull/68/>

Issue L-1: Unused setter instruction [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-03-lombard-solana-contracts/issues/129>

Summary

A `set_treasury` instruction is defined but never used.

Vulnerability Detail

The `mailbox` program, a `set_treasury` instruction is defined but never exposed/used. Thus admins will not be able to change the treasury address if required, even though the instruction exists.

Impact

The admin might be unable to easily set change the treasury address, since the setter isn't exposed.

Code Snippet

<https://github.com/sherlock-audit/2026-03-lombard-solana-contracts/blob/def4cec29853df194a31e1fb69d92ac2572e861c/sol-contracts/programs/mailbox/src/instructions/admin.rs#L53>

Tool Used

Manual Review

Recommendation

Create a callable instruction `set_treasury` allowing the admin to change the treasury address.

Discussion

555-andrew

Acknowledged. Fixed in this pull request:

<https://github.com/lombard-finance/sol-contracts/pull/71>

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.